Project Report: Waste Identification

Last Name	First Name	Student ID
Beatrice	Rebecca	IE22700115
Gaeta	Michela	IE22700116
Genito	Fabio	IE22700145

1 Introduction

This document presents the process of training and performance evaluation of a neural network whose main objective is to categorize waste. The following sections will detail how the team addressed the classification problem step by step.

2 Data Analysis

The first step was to analyze the dataset to understand class distribution and identify under-represented or over-represented classes that could negatively influence the network's learning process.
Using a Python script executed on Google Colab, we generated a graphical representation of the data distribution (Figure 1).

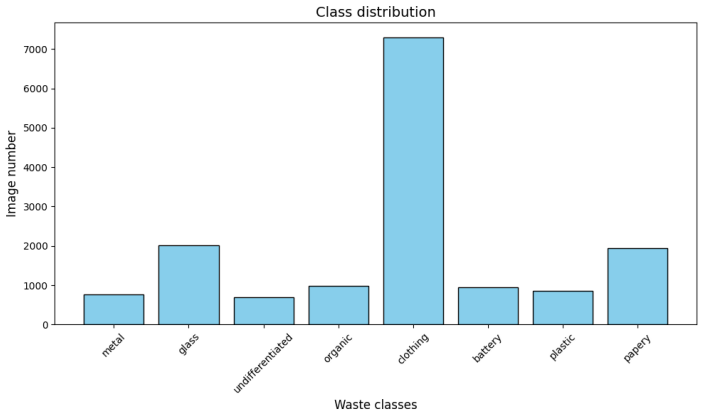


Figure 1: Class distribution in the whole dataset

As shown in Figure 1, a severe imbalance affects the dataset: "Clothing" is heavily over-represented (7 302 images), while other classes, like "Undifferentiated" (697 images) are under-represented. Without any appropriate manipulation, this imbalance in the dataset could make the network categorize almost any waste image as "Clothing" simply because it is the most frequent class. This phenomenon, known as accuracy paradox, would drastically decrease the network's generalization capabilities. To prevent misclassification two strategies were implemented:

- **Targeted data augmentation**: Geometric and chromatic transformations were applied exclusively to the training set, scaling inversely to class size to balance cardinality and mitigate overfitting. These transformations are applied only to the training data; the validation set remains untouched to ensure unbiased performance evaluation.

- **Weighted Loss Function (WCE)**:The loss function was modified to heavily penalize misclassifications on minority classes:

$$L_{WCE} = -\sum_{i=1}^{N} w_i \cdot y_i log(\hat{y}_i)$$

Where w_i counterbalances class frequencies.

3 Analysis of Usable Networks

To achieve accurate classification and high performance, we applied *Transfer Learning* from pre-trained architectures. This approach requires minimal architectural modifications to adapt the model for our waste classification problem. Before initiating the training phase, four models were evaluated, to select the most suitable one, balancing capacity, computational constraints (5GB of GPU RAM), and inference efficiency:

- **YOLOv5**: It is one of the most used networks for Object Detection and belongs to the YOLO (You Only Look Once) family. The network not only performs image feature extraction using a convolutional network, also called *backbone*, but it also generates bounding boxes to localize identified objects using two interacting components known as *neck* and *head*.
  Discarded: Since the main objective of this project is pure Image Classification, using YOLOv5 would introduce complex layers dedicated to spatial localization, resulting in unnecessary and computationally expensive mathematical operations.

- **Trash-Net**: It is a specialized network for waste classification, developed by fine-tuning Google's SigLip2. Unlike YOLOv5, Trash-Net does not rely on a convolutional architecture. Instead, it uses a Transformer network to divide the image into smaller pieces, called *patches*, and evaluates how each patch influences the final classification.
  Discarded: Vision Transformers require massive data to prevent overfitting, exceeding our dataset capacity.

- **ResNet-18**: It is an 18-layer convolutional neural network that solves the vanishing gradient problem using *Skip Connection* and residual mapping. Among all the architecture belonging to the ResNet family, ResNet-18 is the smallest and lightest model. The model uses a ReLU activation function and fully connected layers.
  Selected as the baseline.

- **EfficientNet-B0**: Stands out for its use of *Mobile Inverted Bottleneck Convolution blocks* that dynamically recalibrate channel-feature dependency, capturing fine micro-textures at a fraction of the computational cost.
  Selected for advanced optimization.

3.1 Baseline: ResNet-18

Based on our preliminary analysis, we adopted ResNet-18 pre-trained on the ImageNet database as our baseline model. To exploit the rich feature representations learned on massive image databases, we leveraged **Transfer Learning** with weights pre-trained on *ImageNet*. The final classification head was customized to output $N = 8$ dense neurons mapped via a **Softmax** activation function. What makes ResNet different from the traditional Convolutional Neural Networks (CNNs) is the internal organization of its layers.
ResNet models are built upon a series of basic blocks that consists of two convolutional layers and a

skip connection that preserves the information and makes the gradient stable during the training phase. Mathematically, the network never learns a direct mapping $H(x)$, but rather a residual function:

$$F(x) = H(x) - x$$

So, if a block extracts no useful features, the information passes through the skip connection to the next layer. The complete macro-architecture of our ResNet-18 baseline is structured as follows (Figure 2 and Figure 3):

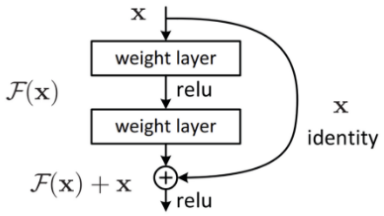


Figure 2: ResNet Block structure

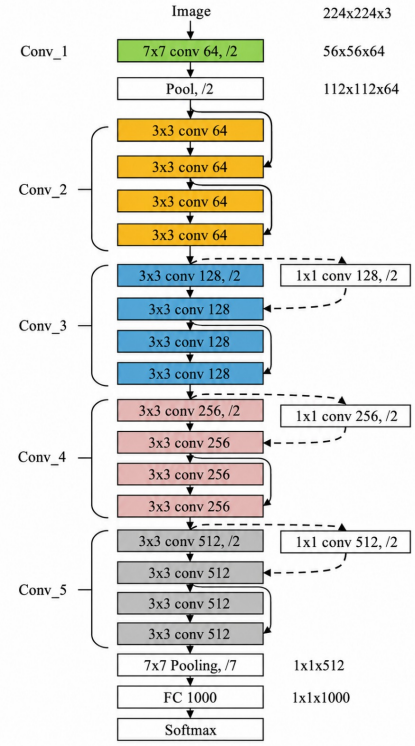


Figure 3: ResNet18 architecture

4 Training Strategy and Hyperparameters

Experiments were conducted on Google Colab (**NVIDIA T4 GPU**). Given the severe class imbalance, the global evaluation metric selected was **Balanced Accuracy**, computed as the unweighted arithmetic mean of the *True Positive Rate (TPR)* or *Recall* across all N categories:

$$TPR_i = \frac{TP_i}{TP_i + FN_i} \quad \Longrightarrow \quad \text{Balanced Accuracy} = \frac{1}{N} \sum_{i=1}^{N} TPR_i$$

To avoid unnecessarily long training and prevent overfitting, this was paired with a dynamic learning rate scheduler (*ReduceLROnPlateau* scheduler) and **Early Stopping**, promoting faster and more stable model convergence.

Hyperparameter	Tested Values / Range	Optimization Objective
Backbones	ResNet-18	Evaluate precision vs inference trade-off
Fine-Tuning Depth	[Full Network, 3-Layers unlocked, 2-Layers Unlocked, Only last layer]	Balance custom feature mapping vs overfitting
Learning Rate (α)	[0.01, 0.001, 0.0001, 10^{-5}]	Control convergence velocity and stability
Batch Size	[16, 32, 64]	Mitigate gradient noise and VRAM occupation
Weight Decay (L_2)	[10^{-5}, 10^{-4}, 10^{-3}]	Apply structural regularization on deep weights
LR Scheduler	ReduceLROnPlateau	Adaptive stepping on validation plateaus
Scheduler Factor	[0.1, 0.5]	Fine-tune learning rate reduction damping
Scheduler Patience	[2, 3] epochs	Set tolerance before decreasing learning rate
Early Stopping	5 epochs	Halt execution to guarantee generalization
Dropout Rate	[0.3, 0.5, 0.7]	Prevent co-adaptation in classification head
Loss Function Weights	[Pure Inverse, Smoothed, Square Root, Class-Balanced Loss]	Calibrate penalties for minority categories
Geometric Transforms	[Rotation, Horizontal Flip, Vertical Flip]	Invariant classification for orientation changes
Color Transforms	[Color Jitter, Brightness, Contrast]	Contrast specular reflections on Glass/Metal

Table 1: Comprehensive Hyperparameter Search Space for Team Optimization

Our empirical process was structured around three main thematic areas of optimization, conducted in parallel to evaluate different aspects of the network's behavior. Before branching into specific experimental configurations, all model were initialized using a shared Baseline Setup (2). This default starting point provided a reference to evaluate all subsequent modifications.

Hyperparameter	Baseline Value
Optimizer	Adam
Initial Learning Rate (α)	0.01
Batch Size	32
Loss Function	Weighted Cross-Entropy
LR Scheduler	ReduceLROnPlateau (factor: 0.1, patience: 2)
Early Stopping Patience	5 epochs

Table 2: Common Baseline Training Configuration

Thanks to this baseline configuration, the model achieved a Balanced Accuracy of 95.24%, a Precision of 95.06% and a F1-Score of 0.9514, converging at epoch 13, where the minimum validation loss reached a value of 0.1589.

4.1 Experiment 1: Regularization and Loss Optimization

These series of experiments focused on mitigating overfitting and calibrating the severity of class penalities:

- **Weight Decay (L2 Regularization):** With a low value, the network fits the training data well but risks overfitting, while a balanced value acts as a regularizer, improving generalization and reducing the gap between training and validation. A too high value forces the weights toward zero, drastically limiting the network's capacity and causing underfitting.
  - *Observations:* Testing showed that a lower penalty (10^{-5}) led to loss divergence. Increasing it to 10^{-3} maximized regularization, minimized validation loss, and stabilized the model against overfitting without sacrificing capacity.

- **Dropout Variation:** With a low dropout rate, the lack of neuron deactivation risks overfitting, while a medium value shuts down nodes optimally, forcing the network to learn robust and generalizable features. A high value deactivates too many connections simultaneously, depriving the model of necessary information and causing underfitting.
  - *Observations:* An aggressive rate of 0.7 in the classification head proved most effective on the Baseline, unlocking the highest Balanced Accuracy on complex classes (plastic/organic) compared to 0.3 and 0.5. However, using the best hyperparameters, the best dropout rate proved to be 0.5.
- **Loss Weight Calibration:** The standard weighted loss or square loss is expected to penalize minority class errors more heavily risking gradient instability. The Class- Balanced Loss approach should optimize this by weighting samples based on effective data volume, ensuring an ideal trade-off for a more stable and robust convergence.
  - *Observations:* Pure inverse frequency weights degraded majority class performance. Square Root Smoothing and Class-Balanced Loss approaches minimized validation loss, providing the best trade-off by stabilizing gradients and boosting overall macro precision.

4.2 Experiment 2: Learning Dynamics and Hyperparameters Tuning

These experiments aimed at identifying the optimal balance between convergence speed and stability:

- **Initial Learning Rate**: Higher values enable larger steps for faster convergence, though they introduce instability and oscillations. On the other hand, lower values ensure precise convergence but require significant computational time and risk stagnation in local minima.
  - *Observations:* A high rate (0.01) caused severe loss oscillations, whereas 10^{-5} led to sub-optimal local minima after an excessive amount of computational time. An intermediate rate (10^{-4}) ensured smooth, precise convergence in a reasonable timeframe.
- **Learning Rate Scheduler & Batch Size**: These parameters manage temporal adaptation and gradient estimation quality. A dynamic reduction strategy refines convergence, while the batch size controls the trade-off between gradient noise, hardware efficiency, and generalization capability.
  - *Observations:* A drastic reduction factor (0.1) paired with a low patience (2) proved to be the most robust choice; whereas a smoother decay led to a degradation in performance. Regarding batch sizes, a size of 32 provided the optimal balance between gradient stability, hardware utilization, and generalization capability.

4.3 Experiment 3: Fine-tuning depth and advanced augmentation

This block targeted feature extraction capabilities and representation invariance using different unfreezing depths and advanced transforms.

- *Observations:* Unfreezing only Layer 3 yielded a solid baseline (96.65% Global Accuracy) but suffered from low precision on reflective materials like Metal (0.77). Switching to Full Fine-Tuning with a lower learning rate (10^{-5}) and adding customized Data Augmentation (removing aggressive brightness changes) successfully eliminated cross-material confusion.

4.4 Optimized ResNet-18 Model

The optimized ResNet-18 achieved a Balanced accuracy of 96.67% and a F1-Score of 0.9651 converging at epoch 20, where the minimum validation loss reached a value of 0.0772. Using Class-Balanced Loss effectively solved the accuracy paradox, achieving high F1-scores on critical minority classes like Plastic (0.94) and Metal (0.94) (Specific performance metrics are shown in Table3).

Waste Class	Precision	Recall	F1-Score
Battery	0.98	0.96	0.97
Clothing	0.99	0.99	0.99
Glass	0.97	0.96	0.97
Metal	0.91	0.97	0.94
Organic	0.96	0.99	0.97
Papery	0.97	0.96	0.96
Plastic	0.95	0.92	0.94
Undifferentiated	0.98	0.97	0.97

Table 3: Best ResNet-18 performance per class

5 Architectural Transition and Advanced Optimization: EfficientNet-B0

Despite the excellent performance of ResNet, it was hypothesized that its rigid structure limited further margins of improvement. Therefore we shifted by migrating to the EfficientNet-B0 architecture (Shown in Figure 5). This model replaces the standard ReLU activation function in its intermediate layers with *Swish* (also known as SiLU-Sigmoid Linear Unit): $SiLU(x) = x \cdot \sigma(x)$.
While ReLU presents a sharp angle in zero, the Swish (or SiLU) function has a smoother curve, that allows small negative values before asymptoting to zero. This enables the gradient to go smoother in the backpropagation phase and mitigates the dying ReLU problem as shown in figure 4.

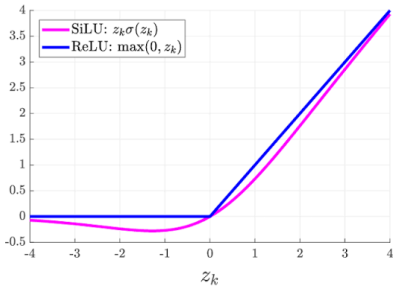


Figure 4: SiLU derivative vs ReLU derivative

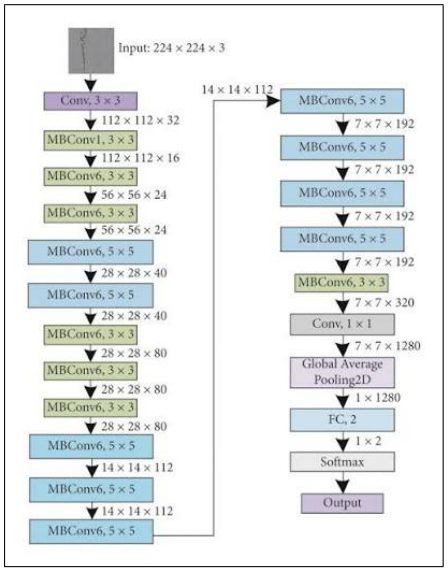


Figure 5: EfficientNet-B0 network architecture

The MBConv (Mobile Inverted Bottleneck Convolution - Figure 6) block is the core building block of EfficientNet. It relies on an inverted bottleneck architecture that optimizes both efficiency and accuracy. The flow begins with a 1×1 convolution that expands the input channels into a high dimensional space, followed by a Depthwise Convolution for efficient feature extraction. A Squeeze-and-Excitation module is applied to recalibrate the features by weighting their relative importance. The tensor is then compressed back to a lower dimensionality with a 1×1 convolution, which acts as a linear bottleneck to prevent information loss. Finally, if input and output dimensions match, a residual skip connection bridges the ends of the block, improving gradient flow.

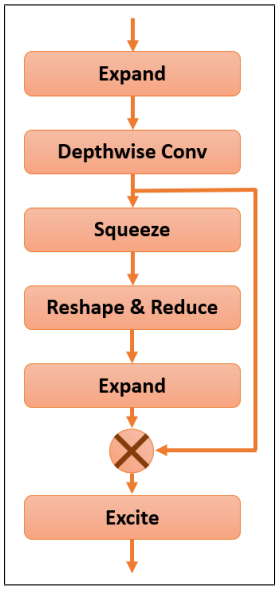


Figure 6: EfficientNet-B0 block architecture

As an initial baseline, the hyperparameters previously optimized for ResNet (Learning Rate = 0.0001, Batch Size =32, Dropout=0.5, Class-Balanced Loss) were applied directly to EfficientNet-B0. The direct transfer of the optimized ResNet hyperparameters immediately boosted the Balanced Accuracy to 96.17% without further tuning, confirming the superior feature extraction capability of MBConv blocks. To exploit the full potential of EfficientNet-B0, we conducted a second targeted optimization phase, that targeted: Regularization and Loss Optimization, Learning Dynamics and Hyperparameters Tuning and Fine-Tuning Depth.
In order to prevent overconfidence we introduced another regularization technique, Label Smoothing, combining it with our existing methods (Weighted Cross Entropy, Dropout etc..). By default, in classification problems, we use one-hot encoded labels, where the correct class is labeled as 1 while the others as 0. This absolute certainty can make the model overconfident, pushing its weights to extreme values and leading to overfitting.
Label Smoothing softens this "hard" labels by subtracting a small fraction of the probability from the correct class and distributing it uniformly among the incorrect classes:

$$y^{LS} = (1 - \alpha) \cdot y + \frac{\alpha}{K}$$

Where:

- y is the original target (0,1)
- α is the smoothing parameter
- K is the total class number (8 in our case)

5.1 Experiment 1: Fine-Tuning Depth and Loss Optimization

This block targeted feature extraction capabilities and representation invariance using different unfreezing depths, while simultaneously evaluating the regularization effects of cross-entropy loss variations through the integration of label smoothing.

- **Loss Weighted Calibration (Label Smoothing)**: Analyzes the integration of standard cross-entropy regularization via label smoothing to prevent excessive probability flattening and overfitting on fine background details.

- *Observations*:Testing three different hyperparameter values on a standardized baseline (Batch Size = 32, Learning Rate = 0.0001, Weight Decay = 0.001, 5 layers unlocked, and Clean Data Augmentation), the configuration with Label Smoothing = 0.1 yielded the best overall trade-off. The training process stopped at epoch 23 via early stopping with a Validation Loss of 0.7714, achieving the highest metrics of the batch, including a Global Accuracy of 97.91% and a Balanced Accuracy of 97.41%. The more pronounced smoothing configuration at 0.15 (stopping at epoch 22) showed a slight performance dip with a Global Accuracy of 97.33%, a Macro F1-score of 0.9593, and lower precision on the critical metal class (0.84), alongside a predictably higher Validation Loss (1.0254). Finally, the experiment with reduced smoothing at 0.05 showed the fastest convergence in 17 epochs with the lowest overall Validation Loss (0.4819), yet it plateaued at a Global Accuracy identical to the previous test (97.33%), while sustaining a solid Macro F1-score of 0.9668. In conclusion, the analyzed data confirms that the intermediate value of 0.1 effectively prevents both under-regularization and excessive probability flattening, proving to be the ideal trade-off for the overall robustness of the model.

- **Evaluation of Unfreezing Depths**:While ResNet required a full fine-tuning approach, EfficientNet demonstrated remarkable baseline efficiency.

  - *Observations*: Working under the same hyperparameter conditions, a fully unlocked EfficientNet achieved a Balanced Accuracy of 97.11%, outperforming the ResNet benchmark. This was accomplished despite EfficientNet possessing less then half of the total parameters of ResNet. However, this state was prone to structural contamination, misclassifying plastic samples as glass. Pushing the freezing too deep (leaving only 4 layers unlocked) caused an accelerated convergence, triggering early stopping after a few epochs. The network suffered from severe underfitting, with a low balanced accuracy, falling below the ResNet benchmark. Freezing the first two convolutional blocks (7 layer unlocked) pushed the model to a peak balanced accuracy of 97.26%. Alternatively, the 5 layers unlocked configuration, with the label smoothing, offered maximum operational efficiency, converging in only 8 epochs and reaching a 97.45% Balanced Accuracy. This setup solved the false-positive issue on the Metal class, pushing its precision to 0.94 without requiring external data augmentation.

5.2 Experiment 2: Learning Dynamics and Hyperparameters Tuning

The objective of this experimental phase was to identify the optimal balance between convergence speed and stability for the EfficientNet architecture, subjecting it to a fine-tuning process.

- **Initial Learning Rate**: The initial learning rate confirms itself as the hyperparameter with the greatest impact on training dynamics. The evidence gathered on EfficientNet mirrors the behavior observed on ResNet.

  - *Observations*: Similar to ResNet's worst-case scenario, an excessively high initial learning rate (0.001) produced a highly unstable Validation Loss. Besides prolonging the training time, this configuration triggered overfitting around epoch 15: from this point onwards the Validation Loss began to degrade while Training Loss continued to decrease. As noted on ResNet, for rates in the order of 10^{-5} , an overly conservative update step generated a very stable descent, with the validation loss following the training loss. In this configuration, the training required the maximum amount of time, taking 55 minutes, and returned the sub-optimal metrics. In perfect analogy with the 10^{-4} rate rewarded on ResNet, the value 0.0001 proved to be the ideal balance. The loss acquired a clean and fluid profile, converging with much more stability. This resulted in achieving the maximum predictive performance.

- **Learning Rate Scheduler**: The Scheduler's patience was tested once again, with results that diverged from the ones obtained with ResNet.

  - *Observations*: In the observations made on ResNet,a reduced tolerance proved to be the most robust choice for instability. Applying the same parameter to EfficientNet resulted in an evident degradation of performance. By increasing the scheduler's tolerance window to 3 epochs, the model expressed its maximum structural potential. While ResNet required an "aggressive" decay (patience 2) to rapidly stabilize gradients in the event of a plateau, the EfficientNet architecture requires more "breathing space". A slightly wider plateau observation window (patience 3) proves critical: a premature intervention to decay the learning rate on EfficientNet traps the weights in early sub-optimal convergences, penalizing the final accuracy and generating, severe computational bottlenecks during training.

5.3 Experiment 3: Optimization of the regularization mechanisms

Since the EfficientNet architecture already includes effective native regularization mechanisms, the objective of these tests is to identify the minimum effective level of regularization required to optimize the results.

- **Weight Decay**: Through a comparative analysis of the optimization tests, it is evident that reducing the number of unlocked layers during fine-tuning alongside a proper calibration of the weight decay significantly enhanced the model's generalization capabilities, systematically outperforming the baseline metrics achieved by the standard ResNet architecture.

  - *Observations*: Within the 7-unlocked-layer configuration, the optimal weight decay value was found to be 0.0001 (Validation Loss: 0.0902, Macro F1-Score: 0.9662), whereas an excessive increase to 0.005 induced a clear underfitting phenomenon, dropping the F1-Score to 0.9436 in just 8 epochs. However, the most substantial increase in quality was observed in the 5-unlocked-layer setup with a weight decay of 0.0001, which emerged as the best overall model; this specific combination achieved the lowest Validation Loss (0.0777), the highest Global Accuracy (0.9826), and a Macro F1-Score of 0.9725. When examining individual classes, it is noteworthy that the most critical categories for ResNet, specifically metal and plastic, benefited from this new configuration, with plastic rising to an F1-score of 0.95 and metal reaching 0.96 in the 5-layer model.

- **Dropout Variation**: In perfect analogy with the scenario observed in the weight decay regularization tests, the introduction of a calibrated Dropout rate imposed a more stable and linear convergence of the loss, outperforming the metrics recorded by the baseline ResNet across all configurations.

  - *Observations*: In the tests without Dropout (Rate 0.0), the network showed a curve descent that halted after 12 epochs with a Validation Loss of 0.1061 and a Macro F1-Score of 0.9569, highlighting a partial tendency toward overfitting. An intermediate increase of the Dropout rate to 0.4 mitigated this phenomenon, reducing the minimum loss to 0.0951 in just 10 epochs and raising the Macro F1-Score to 0.9603; however, the optimal balance between predictive performance and efficiency was achieved with a Dropout Rate of 0.2, recording a generalization peak with a Validation Loss of just 0.0898, a Global Accuracy of 0.9791, and a Macro F1-Score of 0.9682 over 17 training epochs. Conversely, the baseline ResNet suffered heavily from a lower adaptive capacity linked to the increase in Dropout. While ResNet struggled to surpass the critical threshold of a 0.89 F1-score on plastic and leveled off at 0.91

on metal, the new model optimized with a 0.2 Dropout raised the F1-score of plastic to 0.94 and stabilized metal at 0.94.

5.4 Optimized model

The best performing model was achieved by unfreezing and fine-tuning the last 5 layers of the network. The training process incorporated uniform (non-targeted) data augmentation across the dataset to enhance generalization. The optimal configuration utilized Weighted Cross Entropy (WCE) combined with Label Smoothing ($\alpha = 0.1$) to compute the loss. Training was conducted with a batch size of 32, an initial learning rate of 0.0001, and a weight decay of 0.0001 to apply L2 regularization. To further mitigate overfitting, a dropout rate of 0.2 was introduced. Additionally, we implemented a learning rate scheduler that reduced the learning rate of 0.1 after a patience of 3 epochs without improvements.
The optimized EfficientNet-B0 achieved a Balanced accuracy of 97.45% and a F1-Score of 0.9695, converging at epoch 16, where the minimum validation loss reached a value of 0.7712, due to label smoothing.(Specific performance metrics are shown in Table 4).

Waste Class	**Precision**	**Recall**	**F1-Score**
Battery	0.96	0.99	0.97
Clothing	1.00	0.98	0.99
Glass	0.97	0.99	0.98
Metal	0.91	0.97	0.94
Organic	0.98	0.99	0.98
Papery	0.97	0.97	0.97
Plastic	0.95	0.92	0.94
Undifferentiated	0.98	0.98	0.98

Table 4: Best EfficientNet-B0 performance per class

6 Conclusions

This project addressed the complex challenge of waste classification by evolving from a solid baseline architecture to a highly optimized, efficient model. The progression of our experiments highlights several key findings regarding data imbalance, architectural efficiency, and the role of fine-tuning depth. While the ResNet-18 baseline provided a robust starting point, the migration to EfficientNet-B0 proved to be a significant upgrade in both predictive power and computational cost. The fundamental achievement of our optimization phase was proving that full-network fine tuning is not always optimal. We discovered that freezing the primary convolutional blocks of EfficientNet acted as a structural regularizer. This constraint forced the network to leverage its pre-trained ImageNet filters for basic textures, reserving its adaptive capacity for material geometries.

Ultimately, the 5-Layer Unlocked EfficientNet-B0 stands as the best model tested in this project. It achieved a peak Balanced Accuracy of 97.45%, outperforming the fully fine-tuned ResNet-18 benchmark while operating with less than half the total parameters. This outcome highlights a fundamental Deep Learning principle: strategic architectural constraints and computational efficiency are often far more effective than brute-force parameterization.